



database.py

This file (database.py) is the **Data Access Layer (DAL)** of the Retail Sales & Profit Management System. It is responsible for creating the database, storing data, retrieving data, updating records, deleting records, and generating summaries.

database.py Documentation

Purpose of the File

This module manages all interactions with the SQLite database (shop.db).

Responsibilities

- Create database tables
- Insert new products
- Update inventory
- Record sales
- Generate sales summaries
- Retrieve reports
- Delete records
- Reset the system

1. Import Statements

```
import sqlite3
import os
from datetime import datetime
```

Purpose

sqlite3

Provides access to SQLite databases.

Used for:

- Creating tables
- Executing SQL queries
- Managing transactions

**os**

Provides operating system utilities.

Used for:

```
os.makedirs()
```

Creates the database directory automatically.

datetime

Used for:

- Today's sales filtering
- Date-based reporting
- Timestamp management

2. Database Path

```
DB_PATH = "data/shop.db"
```

Purpose

Defines where the SQLite database file is stored.

Result:

```
project/
|
|-- data/
|   |-- shop.db
|
|-- app.py
|-- database.py
```



3. init_db()

```
def init_db():
```

Purpose

Initializes the database and creates required tables.

This function runs automatically when the application starts.

Create Data Folder

```
os.makedirs("data", exist_ok=True)
```

Purpose

Creates:

```
data/
```

if it does not already exist.

Database Connection

```
conn = sqlite3.connect(DB_PATH)
```

Creates connection to SQLite.

Cursor Object

```
cursor = conn.cursor()
```

Allows execution of SQL commands.



Products Table

CREATE TABLE IF NOT EXISTS products

Purpose

Stores inventory.

Columns

Column	Purpose
id	Product ID
name	Product name
category	Product category
buying_price	Supplier cost
selling_price	Retail price
quantity	Stock quantity
created_at	Creation date
updated_at	Last modification

Sales Table

CREATE TABLE IF NOT EXISTS sales

Purpose

Stores every transaction.



Columns

Column	Purpose
id	Sale ID
product_id	Product reference
product_name	Product sold
quantity	Quantity sold
unit_price	Selling price
total_amount	Revenue
cost_price	Cost basis
profit	Profit earned
payment_method	Cash/M-Pesa/Card
created_at	Sale timestamp

Daily Summary Table

```
CREATE TABLE IF NOT EXISTS daily_summary
```

Purpose

Designed for future performance optimization.

Stores:

- Daily sales totals
- Daily profit
- Daily transactions

Currently not actively populated.

Save Changes

```
conn.commit()
```

Writes changes to database.



Close Connection

```
conn.close()
```

Releases database resources.

4. get_connection()

```
def get_connection():
```

Purpose

Creates a reusable database connection.

Returns:

```
sqlite3.connect(DB_PATH)
```

Used throughout the application.

5. add_product()

```
def add_product(...)
```

Purpose

Adds a new product into inventory.



SQL Operation

```
INSERT INTO products
```

Inputs

Parameter	Description
name	Product name
category	Product category
buying_price	Cost price
selling_price	Selling price
quantity	Initial stock

Success Response

```
(True, "Product added successfully")
```

Duplicate Product Protection

```
except sqlite3.IntegrityError
```

Triggered when product already exists.

Returns:

```
(False, "Product already exists")
```

6. update_product()

```
def update_product(...)
```

Updates existing inventory records.



SQL Statement

```
UPDATE products
```

Updates:

- Name
- Category
- Buying price
- Selling price
- Quantity

Timestamp Update

```
def get_all_products():
```

Records last modification time.

7. get_all_products()

```
def get_all_products():
```

Purpose

Retrieves all products.



SQL Query

```
SELECT
```

Returns:

Field
id
name
category
buying_price
selling_price
quantity

Sorting

```
ORDER BY name
```

Alphabetical listing.

8. get_product_by_id()

```
def get_product_by_id(product_id)
```

Purpose

Fetches a single product.

SQL Query

```
WHERE id=?
```

Returns one record only.



9. reduce_stock()

```
def reduce_stock(product_id, quantity)
```

Purpose

Automatically decreases stock after a sale.

SQL Logic

```
quantity = quantity - sold_quantity
```

Example:

Current Stock = 50

Sold = 3

New Stock = 47

10. add_sale()

```
def add_sale(...)
```

Purpose

Records completed sales transactions.

This is one of the most important functions.

Revenue Calculation

```
total_amount = quantity * unit_price
```

Example:

$5 \times 100 = 500$



Profit Calculation

```
profit =  
(unit_price - cost_price)  
* quantity
```

Example:

```
Selling = 120  
Cost = 80  
Qty = 5  
  
Profit = 200
```

Insert Sale

```
INSERT INTO sales
```

Stores transaction permanently.

Return Values

```
return total_amount, profit
```

Used by the user interface.

11. get_today_sales()

```
def get_today_sales()
```

Purpose

Returns all transactions made today.



Today's Date

```
today =  
datetime.now().strftime('%Y-%m-%d')
```

SQL Filter

```
WHERE DATE(created_at)=?
```

Returns only today's records.

Sorting

```
ORDER BY created_at DESC
```

Newest sales appear first.

12. get_sales_by_date_range()

```
def get_sales_by_date_range()
```

Purpose

Provides data for reports.

SQL

```
BETWEEN ? AND ?
```

Example:

```
01-Jun-2026  
to  
30-Jun-2026
```

**Used By**

- Weekly reports
- Monthly reports
- Custom reports

13. get_today_summary()

```
def get_today_summary()
```

Purpose

Calculates dashboard KPIs.

SQL Aggregations**Transactions**

```
COUNT(*)
```

Number of sales.

Total Sales

```
SUM(total_amount)
```

Revenue.

Total Profit

```
SUM(profit)
```

Profit.



Items Sold

```
SUM(quantity)
```

Quantity sold.

Output

```
{
  'transaction_count': 10,
  'total_sales': 5000,
  'total_profit': 1500,
  'total_items_sold': 65
}
```

14. delete_product()

```
def delete_product(product_id)
```

Purpose

Removes a product from inventory.

SQL

```
DELETE FROM products
```

Permanent deletion.

15. clear_all_sales()

```
def clear_all_sales()
```

Purpose

Deletes all sales transactions.



SQL

```
DELETE FROM sales
```

Inventory remains intact.

16. clear_all_products()

```
def clear_all_products()
```

Purpose

Deletes all inventory items.

SQL

```
DELETE FROM products
```

Sales history remains.

17. reset_database()

```
def reset_database()
```

Purpose

Performs a full system reset.

Drop Tables

```
DROP TABLE IF EXISTS sales  
DROP TABLE IF EXISTS products  
DROP TABLE IF EXISTS daily_summary
```

Deletes entire database structure.



Rebuild Database

```
init_db()
```

Creates fresh empty tables.

Result

System returns to:

```
Brand New Installation State
```

18. Automatic Initialization

```
init_db()
```

Located at the end of the file.

Purpose

Runs automatically whenever:

```
import database
```

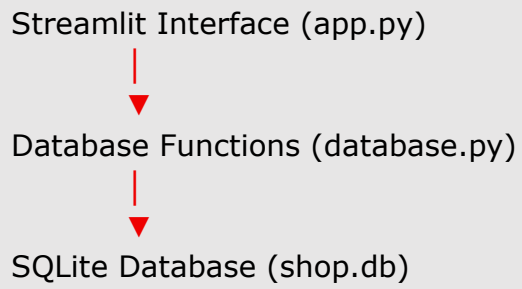
is executed.

This ensures:

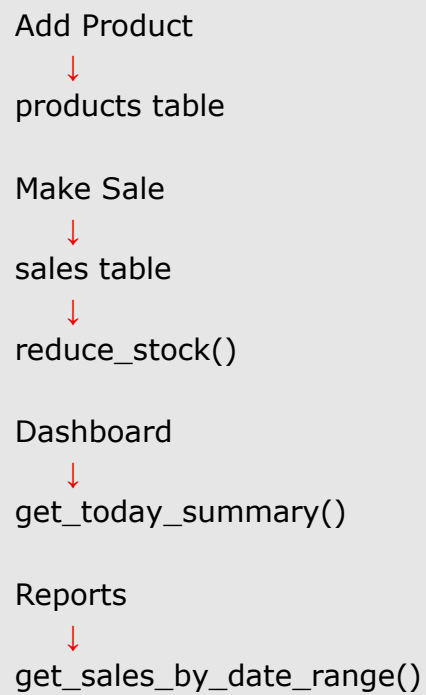
- Database exists
- Tables exist
- Application starts safely



Overall Database Architecture



Data Flow





Functions Summary

Function	Purpose
init_db()	Create database and tables
get_connection()	Open database connection
add_product()	Add inventory item
update_product()	Modify inventory item
get_all_products()	Retrieve inventory
get_product_by_id()	Retrieve one product
reduce_stock()	Deduct sold stock
add_sale()	Record transaction
get_today_sales()	Get today's transactions
get_sales_by_date_range()	Generate reports
get_today_summary()	Dashboard KPIs
delete_product()	Remove product
clear_all_sales()	Delete sales history
clear_all_products()	Delete inventory
reset_database()	Full system reset

This file acts as the **central data management engine** of the entire Retail Sales & Profit Management System. Without it, the Streamlit interface would have no persistent storage, reporting capability, or inventory tracking functionality.